

《动态个人时间与习惯管理系统》

系统设计说明书

研发团队：源浪逐行

2026年5月22日

目录

1	1. 引言	2
1.1	1.1 编写目的	2
1.2	1.2 项目背景	2
1.3	1.3 术语与缩写定义	2
2	2. 总体设计	2
2.1	2.1 运行环境	2
2.2	2.2 系统体系结构（分层设计）	2
2.2.1	2.2.1 表现层 (Presentation Layer)	2
2.2.2	2.2.2 业务逻辑层 (Business Logic Layer)	3
2.2.3	2.2.3 数据访问层 (Data Access Layer)	3
3	3. 接口设计	3
3.1	3.1 用户接口设计	3
3.2	3.2 外部接口设计 (External API)	3
3.3	3.3 内部接口设计 (Internal RESTful API)	3
4	4. 运行设计与出错处理	3
4.1	4.1 运行控制模块	3
4.2	4.2 出错处理设计	4

1 1. 引言

1.1 1.1 编写目的

本系统设计说明书旨在详细阐述“动态个人时间与习惯管理系统（智能时间大脑）”的总体架构、接口规范及运行机制。本文档作为项目开发、测试及后期维护的核心技术指南，主要面向开发人员、测试工程师及项目评审委员会。

1.2 1.2 项目背景

当代大学生面临课业繁重、计划易受突发事件打乱的痛点，传统的静态日程表缺乏柔性及情绪安抚机制。本项目采用微信小程序为载体，引入大语言模型（LLM）作为核心调度器，旨在打造一个具备同理心和动态重组能力的智能时间管理助手。

1.3 1.3 术语与缩写定义

为保证文档表述的一致性与准确性，特对以下缩略语进行预先定义：

- **LLM (Large Language Model)**: 大语言模型，本项目中专指用于解析自然语言并输出结构化指令的 AI 引擎（如 DeepSeek、Kimi 等）。
- **MVP (Minimum Viable Product)**: 最小可行性产品，指本项目第一阶段快速交付的可用版本。
- **API (Application Programming Interface)**: 应用程序编程接口，用于前后端及外部系统的通信。
- **JSON (JavaScript Object Notation)**: 一种轻量级的数据交换格式，本项目用于规范化 AI 的输出与前后端数据传输。
- **UI (User Interface)**: 用户界面，本项目中主要指微信小程序前端视图层。

2 2. 总体设计

2.1 2.1 运行环境

- **客户端环境**: 微信客户端（iOS/Android），支持微信小程序运行基础库 2.15.0 及以上版本。
- **服务端环境**: Linux/Windows 服务器，Python 3.8+ 运行环境。
- **网络环境**: 需具备访问互联网公网的能力，以确保能够调用外部大模型 API 接口。

2.2 2.2 系统体系结构（分层设计）

本系统采用经典的 C/S 变体（微信小程序 + 微服务 API）架构，自上而下分为三层，随着层次加深，处理的业务细节越发具体：

2.2.1 2.2.1 表现层 (Presentation Layer)

基于微信小程序原生框架（WXML/WXSS/JS）构建。负责与用户进行直接交互。功能包括：渲染动态自适应光晕（Status Halo）、展示今日日程列表、捕获用户点击与输入事件、展示系统弹窗与 AI 回复提示框。

2.2.2 业务逻辑层 (Business Logic Layer)

部署于 Flask Web 框架内，是系统运转的核心中枢。功能包括：接收表现层转发的路由请求（如新增、删除、状态反馈）；拼装系统提示词（System Prompt）；调用外部 LLM 控制器进行自然语言理解与 JSON 数据反解析；执行日程数据的合并、排序与冲突校验。

2.2.3 数据访问层 (Data Access Layer)

负责系统状态及用户数据的持久化存储。在 MVP 阶段通过本地 JSON I/O 实现高速读写，在标准生产环境中则映射至 MySQL 关系型数据库，实现用户身份（OpenID）、日程记录的高效查询与持久化保护。

3 接口设计

3.1 用户接口设计

系统遵循极简、无负担的设计理念，摒弃复杂的级联菜单。主要界面接口包括：

- **主控看板**: 采用双分栏视觉逻辑，上部为动态光斑与日程列表，下部为情感交互舱。
- **快捷情境面板**: 预设高频突发场景按钮（如“实验课拖堂”、“临时班会”），实现“一键式”状态注入。
- **自定义表单弹窗**: 采用全屏遮罩 Modal 层，包含三个必填项（时间、任务名、标签），遵循即填即走的交互原则。

3.2 外部接口设计 (External API)

本系统需调用外部大语言模型接口进行智能推演。

- **通信协议**: HTTPS (TLS 1.2)
- **数据封装**: 标准 OpenAI API 协议格式
- **安全机制**: 采用 Bearer Token 鉴权机制。

3.3 内部接口设计 (Internal RESTful API)

前后端通过 HTTP 协议进行通信，采用严格的 JSON 数据格式。主要定义以下接口：

- **获取日程列表**: GET /api/tasks
- **新增日程任务**: POST /api/task/add (参数: time, name, tag)
- **删除日程任务**: POST /api/task/delete (参数: id)
- **AI 智能调度请求**: POST /api/adjust (参数: input)

4 运行设计与出错处理

4.1 运行控制模块

系统的核心生命周期由“突发事件驱动”。当用户发起 /api/adjust 请求时，控制器挂起前端 UI 渲染（展示 Loading 态），拉起服务端通信线程。服务端重组当前数据字典，透传至 LLM。LLM 返回数据后，进行 Schema 结构校验，校验通过则覆盖原始数据层，并返回 200 状态码唤醒前端视图更新。

4.2 4.2 出错处理设计

- **网络超时中断 (Timeout):** 前端设置 15 秒超时阈值。若触发，则提示“系统连接星区网络失败，请检查手机网络设置”，并自动恢复界面可交互状态。
- **模型调用异常 (API Error):** 针对 500 内部错误，服务端捕获异常后，进行平滑降级处理。不再抛出堆栈报错，而是返回系统预设的温情降级 JSON: `{"reply": " ", "hardware_cmd": "RELAX", "updated_tasks": }`，确保系统不死机。